Records were introduced in **C# 9** and improved in later versions (.NET 6/7/8). They are widely used in **modern .NET APIs, CQRS, and immutable models**

---

1. What are Records in C#?

A **Record** is a **reference type designed for storing immutable data**.

It is mainly used for **data models where the value of the object matters rather than the object identity**.

Traditional Class Example

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

Two objects with the same values are **not equal**.

```
var emp1 = new Employee { Id = 1, Name = "John" };
var emp2 = new Employee { Id = 1, Name = "John" };

Console.WriteLine(emp1 == emp2);
```

Output

False

Because **classes compare references**.

Record Example

```
public record Employee(int Id, string Name);
```

Usage

```
var emp1 = new Employee(1, "John");
var emp2 = new Employee(1, "John");
```

```
Console.WriteLine(emp1 == emp2);
```

Output

True

Because **records compare values, not references**.

---

2. Why Records Were Introduced

Records solve problems with **data models**.

Before records we had to write a lot of code.

Records provide:

✓ Value-based equality
✓ Built-in immutability
✓ Built-in ToString()
✓ Non-destructive mutation
✓ Less boilerplate code

---

3. Basic Record Syntax

Positional Record

```
public record Student(int Id, string Name, int Marks);
```

Usage

```
Student s1 = new Student(1, "Ravi", 90);


Console.WriteLine(s1);
```

Output

Student { Id = 1, Name = Ravi, Marks = 90 }

Notice the **automatic ToString() implementation**.

---

4. Record vs Class

Feature	Class	Record
Equality	Reference	Value
Immutability	Manual	Built-in
Boilerplate Code	More	Less
ToString()	Manual	Auto-generated
Use Case	Business logic	Data models

---

5. Record with Properties (Non-Positional)

You can also define records like classes.

```
public record Employee
{
    public int Id { get; init; }
    public string Name { get; init; }
}
```

Usage

```
var emp = new Employee
{
    Id = 101,
    Name = "David"
};
```

Notice

init

This means the property can be **set only during object initialization**.

---

6. Immutability in Records

Records are **immutable by default**.

Example

```
public record Product(int Id, string Name);
```

Usage

```
var p1 = new Product(1, "Laptop");


// p1.Name = "Mobile"; ❌ Not allowed
```

Because records encourage **immutable objects**.

This is very useful in

- Multithreading
- API request/response models

---

7. Non-Destructive Mutation (with Expression)

Records allow you to **create a modified copy without changing the original object**.

Example

```
public record Order(int Id, string Product, int Quantity);
```

Usage

```
var order1 = new Order(1, "Laptop", 2);


var order2 = order1 with { Quantity = 5 };


Console.WriteLine(order1);
Console.WriteLine(order2);
```

Output

Order { Id = 1, Product = Laptop, Quantity = 2 }

Order { Id = 1, Product = Laptop, Quantity = 5 }

Original object **remains unchanged**.

This feature is extremely powerful.

---

8. Records with Methods

Records can contain methods.

Example

```
public record Employee(int Id, string Name, double Salary)
{
    public double AnnualSalary()
    {
        return Salary * 12;
    }
}
```

Usage

```
var emp = new Employee(1, "John", 50000);


Console.WriteLine(emp.AnnualSalary());
```

Output

600000

---

9. Record Inheritance

Records support inheritance.

```
public record Person(string Name);
```

```
public record Employee(string Name, int Id) : Person(Name);
```

Usage

```
Employee emp = new Employee("Ravi",101);


Console.WriteLine(emp);
```

Output

```
Employee { Name = Ravi, Id = 101 }
```

---

10. Record Struct

C# also supports **Record Struct**.

Normally records are **reference types**, but record structs are **value types**.

Example

```
public record struct Point(int X, int Y);
```

Usage

```
Point p1 = new Point(10,20);
Point p2 = new Point(10,20);


Console.WriteLine(p1 == p2);
```

Output

```
True
```

---

12. Compiler Generated Code (Behind the Scenes)

When you create a record:

```
public record Person(string Name);
```

The compiler automatically generates:

- Constructor

- Equals()
- GetHashCode()
- ToString()
- Deconstruct()

Example

```
var person = new Person("John");


var (name) = person;


Console.WriteLine(name);
```

Output

John

This is called **Deconstruction**.

---

13. When to Use Records

Use records when:

✓ Creating DTOs
✓ API request/response models
✓ Immutable objects
✓ Data transfer models
✓ CQRS query results

Avoid records when:

❌ Complex business logic
❌ Mutable entities
❌ Entity Framework entities (usually classes are better)

---

14. Records in Modern .NET Architecture

Records are widely used in:

- **CQRS**
- **MediatR**
- **Minimal APIs**
- **DTO Models**
- **Event Sourcing**

---

15. Practical Demo Program

Complete example:

```
using System;

public record Student(int Id, string Name, int Marks);

class Program
{
    static void Main()
    {
        var s1 = new Student(1, "Ravi", 90);
        var s2 = new Student(1, "Ravi", 90);

        Console.WriteLine(s1 == s2);

        var s3 = s1 with { Marks = 95 };

        Console.WriteLine(s1);
        Console.WriteLine(s3);
```

```
    }
}
```

Output

```
True
Student { Id = 1, Name = Ravi, Marks = 90 }
Student { Id = 1, Name = Ravi, Marks = 95 }
```